

5 GPIO

In dit hoofdstuk behandelen we GPIO (General-Purpose Input/Output), de eenvoudigste periferie die aangestuurd kan worden.

5.1 Output

5.1.1 Basisinstellingen

Na het resetten van de microcontroller zijn alle pinnen standaard ingesteld als input. Dit voorkomt schade aan de hardware, omdat een input zich als een hoogimpedante verbinding gedraagt en daardoor geen grote belasting vormt voor aangesloten componenten.

Om een pin als output te gebruiken, moeten we enkele instellingen aanpassen. De eerste stap is uiteraard bepalen welke pin we willen gebruiken. Hiervoor raadplegen we, afhankelijk van de toepassing, de **beschikbare documentatie** zoals Nucleo-schema, Nucleo-bordafbeeldingen op Toledo of Nucleo Extension Shield schema.

De pinnen worden steeds aangeduid met de letter P gevolgd door de poortnaam (A t.e.m. F) en het pinnummer (0 t.e.m. 15).

Let op: niet alle pinnen van de microcontroller zijn **extern toegankelijk** via het Nucleo-bord!

Stel dat we LED1 op het Nucleo Extension Shield willen aansturen, zullen we eerst moeten opzoeken op welke pin deze LED is aangesloten om als output in te stellen. Uit het schema blijkt dat LED1 op pin PC0 zit.

Vervolgens kan deze pin als output worden ingesteld met de volgende code:

```
// Clock voor GPIOC inschakelen.  
RCC->AHBENR = RCC->AHBENR | RCC_AHBENR_GPIOCEN;  
  
// LED1 op output zetten  
GPIOC->MODER = (GPIOC->MODER & ~GPIO_MODER_MODER0) | GPIO_MODER_MODER0_0;
```

Figuur 22: codefragment voor het instellen van LED1

De gebruikte benamingen zijn gedefinieerd door de fabrikant in de CMSIS-bibliotheek.

Zo is *AHBENR* (*AHB peripheral clock enable register*) een register binnen de *RCC* (*Reset and Clock Control*) periferie en *MODER* (*GPIO port mode register*) een register binnen de *GPIOC*-periferie (poort C van *GPIO*).

In de code wordt een register bereikt d.m.v. deze constructie: *periferienaam->registernaam*

De betekenis van de *->* operator wordt later uitgelegd.

Om de exacte betekenis en mogelijke instellingen van deze registers te achterhalen, dient de datasheet *STM32F0 Reference Manual - DM00031936* geraadpleegd te worden. In deze cursus zullen we dit regelmatig doen.

We navigeren hierin naar het hoofdstuk over de specifieke periferie (RCC en vervolgens GPIO).

Binnen elk hoofdstuk staat in de laatste paragraaf een oplijsting van de registers in tabelvorm met de nodige uitleg.

Elk register is 32-bits breed waarin elke bit een betekenis heeft, maar niet alle bits worden altijd gebruikt (ze worden dan vaak als *reserved* aangeduid).

Merk op dat we in voorgaande code binaire operatoren gebruiken om bitwijzigingen door te voeren. De binaire operatoren zijn: | & ~ ^ >> <<

In de **eerste stap** zullen we de klok voor poort C gaan inschakelen:

```
RCC->AHBENR = RCC->AHBENR | RCC_AHBENR_GPIOCEN;
```

Hierin wordt de | operator (OR) gebruikt om specifieke bits in *AHBENR* in te stellen zonder de andere bits te beïnvloeden.

Dit wordt vaak het **zetten van bits** genoemd.

RCC_AHBENR_GPIOCEN is echter een macro (constante) die in het bestand *stm32f091xc.h*, na wat zoeken, gedefinieerd is als $(1 << 19)$.

De code kan dus eventueel ook geschreven worden als *RCC->AHBENR = RCC->AHBENR | (1 << 19);*

Hierin stelt de << de linker schuifoperator (left shift operator), dit zorgt ervoor dat de waarde '1' gezet wordt gevolgd door negentien keer '0', m.a.w. op bitpositie 19 (te tellen van bit 0) zetten we een 1.

Deze bit betekent volgens de datasheet "*I/O port C clock enabled*". Als we dit niet zouden inschakelen, zal poort C gewoon niet werken als we hierop één of meerdere pinnen willen gebruiken.

Opmerking: i.p.v. de notatie $(1 << 19)$, kunnen we ook de notatie *0x80000* (hexadecimaal) of gewoon *524288* (decimaal) gebruiken in de code, maar dit maakt de code moeilijker om te controleren.

We moeten namelijk steeds terug omrekenen naar binaire waarden om te zien welke bits al dan niet gezet zijn geweest in een register.

(Verder bestaat ook de binaire notatie dat in code als *0b10000000000000000000* geschreven kan worden, maar dit is zeer sterk afgeraden! Het wordt al snel omslachtig, want voor sommige waarden zouden we al tot 32-bits moeten neerschrijven. Deze notatie gaan we bijgevolg niet gebruiken!)

In de **tweede stap** zullen we pin PC0 als output zetten:

```
GPIOC->MODER = (GPIOC->MODER & ~GPIO_MODER_MODER0) | GPIO_MODER_MODER0_0;
```

We werken met *PC0*, dus dienen we in *MODER* onder *MODER0*[1:0] twee bits in te stellen om de pin als output te configureren door de waarde '01' in te stellen.

We vervangen de macro's even door hun werkelijke waarden:

```
GPIOC->MODER = (GPIOC->MODER & ~(3 << 0)) | (1 << 0);
```

Merk op dat $<< 0$ weggelaten kan worden, want dit zal niets doen, echter maakt het de code duidelijker dat de waarde 3 (binair '11') vanaf bitpositie 0 wordt geplaatst en de waarde 1 op bitpositie 0.

Het eerste deel van de bewerking (*GPIOC->MODER & ~(3 << 0)*) zorgt ervoor dat 3 (= binair dertig keer '0' gevolgd door '11') geïnverteerd wordt d.m.v. de ~-operator. We verkrijgen dan als resultaat binair dertig keer '1' gevolgd door '00'. (*)

Daarna wordt deze waarde met de &-operator toegepast op *GPIOC->MODER*. Dit zorgt ervoor dat de onderste twee bits op '00' worden gezet. Dit proces wordt vaak het **maskeren van bits** genoemd.

Het tweede deel van de bewerking *| (1 << 0)* zal het voorgaand resultaat combineren met een '1'.

Het uiteindelijke resultaat is binair '01' wat we wegschrijven naar *MODER*.

(*) Opmerking: het is in ons codevoorbeeld optioneel om expliciet '00' te gaan wegschrijven in *MODER* vermits '00' al de resetwaarde is. Maar het is een goede gewoonte om dit toch te doen, het

kan best zijn dat de pin voorheen in de code eerst een andere modus had. Of, in sommige toepassingen, kan eenzelfde pin hergebruikt worden en wordt er voortdurend geswitcht tussen input en output.

5.1.2 Maximumstroom

Elke IO mag maximum 25 mA leveren (source) of opnemen (sink). De som van alle stromen in de microcontroller mag niet hoger zijn dan 80 mA. Die info kan teruggevonden worden in de datasheet van de STM32F091RC.

5.1.3 Schakelsnelheid

Om het energieverbruik van de microcontroller zoveel mogelijk te beperken kunnen de stijg- en daaltijden van de outputs ingesteld worden. Hoe sneller een output moet schakelen, hoe meer energie daarvoor nodig is. De schakelsnelheid wordt ingesteld via het *OSPEEDR* register.

Vragen:

- Zoek uit hoe dit ingesteld staat na reset en bekijk de maximum schakelsnelheden als we een voedingspanning van 3,3 V gebruiken.

5.1.4 Output aansturen

Vul hieronder aan hoe we LED1 zouden kunnen in- en uitschakelen:

```
GPIOC->ODR =
```

```
GPIOC->ODR =
```

Figuur 23: codefragment voor het in- en uitschakelen van LED1

Vragen:

- Analyseer de neergeschreven code en verklaar elk deel.

5.1.5 Output toggelen

Als kort maar krachtig bewijs dat de klokinstellingen (zie later voor meer info) en de outputs van de microcontroller correct zijn ingesteld, kunnen we een knipper-LED maken. Hier nemen we terug LED1 als voorbeeld.

Een eerste manier is d.m.v. de LED in te schakelen, even te wachten (delay), de LED terug uit te schakelen, terug even te wachten (delay) en deze cyclus te herhalen.

Een tweede manier is via deze code gevolgd door een delay:

```
GPIOC->ODR = GPIOC->ODR ^ GPIO_ODR_0;
```

Figuur 24: codefragment voor het knipperen van LED1

Vragen:

- Wat doet de ^ operator?

Hoe kunnen we nu een vertraging (**delay**) maken zodat we de LED effectief zien knipperen? Er zijn daar verschillende manieren voor, zowel softwarematig als hardwarematig.

Hier bespreken we de softwarematige manier, eigenlijk is dit niet anders dan een lus (of soms zelfs twee geneste lussen) met een hoge tellerwaarde zodat de microprocessor een tijdje zal blijven lussen alvorens verder te gaan met code.

Een eerste aanzet is om bv. deze lus te creëren:

```
// ... code ...

// Een tijdje wachten
for(ticks = 0; ticks < 2000000; ticks++)
{
    __NOP();
}

// ... code ...
```

Figuur 25: codefragment delay

Vragen:

- Wat betekent `__NOP()`? Hoelang duurt het uitvoeren van deze regel?
- Hoelang duurt het uitvoeren van de for-lus?
- Wat is het nadeel van deze delay?
- Welk datatype kiezen we best voor de variabele `ticks`?

5.2 Input

5.2.1 Basisinstellingen

Zoals eerder vermeld, zijn na het resetten van de microcontroller alle pinnen standaard ingesteld als input. We kunnen die dus onmiddellijk gebruiken; het resetten van `MODER` door '00' op de juiste bitpositie weg te schrijven is in feite niet nodig, tenzij de pin voordien een andere modus had. Wat wel nog steeds nodig is, is het activeren van de klok voor de poort.

Stel dat we SW1 op het Nucleo Extension Shield willen inlezen, bekijken we in het schema op welke pin deze knop is aangesloten. Vervolgens schrijven we de code om de klok te voorzien:

```
// SW1 is standaard input na power on reset, wel nog klok voorzien.
RCC->AHBENR = RCC->AHBENR | RCC_AHBENR_GPIOAEN;
```

Figuur 26: codefragment voor het voorzien van een klok op poort A

5.2.2 Pull-up/pull-down

Om de ontwerper gemakkelijk inputs te laten verwerken, voorziet de microcontroller de mogelijkheid om interne pull-up of pull-down weerstanden te gebruiken. De instelling gebeurt via `PUPDR`.

Vragen:

- Zoek uit wat de instelling is na reset in de datasheet.

Het Nucleo Extension Shield heeft externe pull-up weerstanden *on board*. De interne weerstanden van de STM32F091RC zullen we dus in dit geval niet gebruiken.

5.2.3 Input inlezen

Als basisvoorbeeld van het verwerken van input gaan we de toestand van SW1 inlezen en op LED1 weergeven:

```
if ((GPIOA->IDR & GPIO_IDR_1) != GPIO_IDR_1)
    GPIOC->ODR = GPIOC->ODR | GPIO_ODR_0;
else
    GPIOC->ODR = GPIOC->ODR & ~GPIO_ODR_0;
```

Figuur 27: codefragment inlezen van SW1

Vragen:

- Ontleed de code en verklaar elk stuk. Bekijk welke waarde de macro `GPIO_IDR_1` is en waarom het if-statement zo opgebouwd is?
- Hoe kan men weten als een knop een opgaande (*rising edge*) of neergaande flank (*falling edge*) heeft?
 - Maak een extra hulpvariabele aan die de vorige toestand van de knop bijhoudt.

Opmerking: de bovenstaande code wordt voortdurend uitgevoerd en bevindt zich, al dan niet rechtstreeks, in de oneindige while-lus. De if-conditie wordt dus altijd uitgevoerd om na te gaan of de knop is ingedrukt. Dit principe wordt **polling** genoemd.

De code is eenvoudig en vereist niet veel instellingen, maar is niet efficiënt. Een deel van de rekentijd van de microprocessor wordt besteed aan het controleren van dergelijke periferie. Er bestaan betere technieken, zoals het gebruik van **interrupts**. Zie verder.

6 Basisklokinstellingen

Voordat we verdergaan met bv. andere periferie, is het goed om eerst even stil te staan bij de klokinstellingen.

6.1 Klok

De microcontroller kan alleen zijn werk doen (instructies uitvoeren) als hij wordt gevoed met een kloksignaal. De klok zorgt ervoor dat de microcontroller een instructie start en afhandelt, waarna de volgende instructie volgt, enzovoort.

Ook periferie zoals de ADC, timers en seriële interfaces (UART, I²C, SPI) zijn hiervan sterk afhankelijk en werken alleen correct als de klok goed is ingesteld.

Hoe sneller de klokpulsen, hoe sneller de microcontroller werkt. Uiteraard is er een bovengrens aan die snelheid. Die bovengrens wordt onder andere bepaald door parasitaire capaciteiten in de printbanen, de imperfecties van de microcontroller-chip ('die'), en het maximaal toegestane energieverbruik.

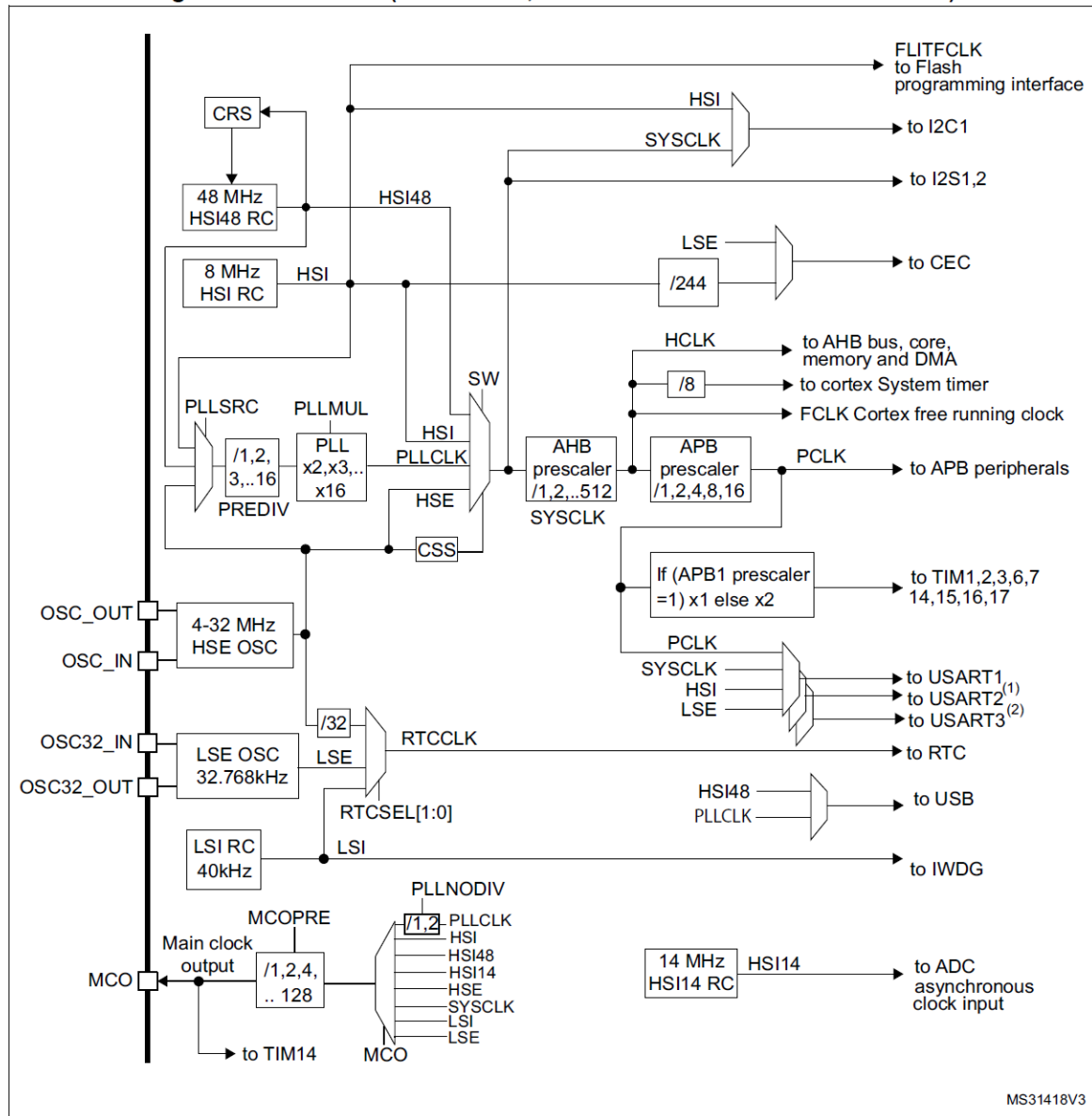
Een microcontroller kan prima functioneren op een klokfrequentie die lager is dan zijn maximale werkbare frequentie. Een hogere frequentie dan toegestaan moet echter wel worden vermeden. Het is aan de ontwerper van het systeem of de software om de juiste klokinstelling te bepalen.

6.2 Kloksysteem

Omdat het instellen van de klok soms vrij ingewikkeld kan zijn, voorzien de meeste fabrikanten een visuele voorstelling van het hele kloksysteem, de zogenaamde **clock tree**.

De *clock tree* op de volgende pagina is heel belangrijk! Zonder een (juiste) klok, geen (correct werkende) microcontrollers...

Figure 12. Clock tree (STM32F04x, STM32F07x and STM32F09x devices)



1. Not available on STM32F04x devices.

2. Not available on STM32F04x and STM32F07x devices

FCLK acts as Cortex[®]-M0's free-running clock. For more details refer to the *ARM Cortex[™]-M0 r0p0 technical reference manual (TRM)*.

Figuur 28: clock tree STM32F091RC (bron: RM0091)

Vragen:

- Zoek uit langs welk pad we de CPU-kern met **48 MHz** zouden kunnen voeden.
Let op: gebruik niet de HSI48 RC klok, want die is niet stabiel genoeg en wensen we niet te gebruiken.

6.3 Instellingen

Op het moment dat de microcontroller gereset wordt, wordt gebruik gemaakt van een basisklokinstelling. Onze gebruikte microcontroller heeft intern een HSI (High Speed Internal) RC oscillator van 8 MHz, waardoor een extern kristal niet echt hoeft gebruikt te worden (tenzij we een heel stabiele klok wensen te hebben).

Die HSI RC klok werkt dus op 8 MHz. Die 8 MHz kan via **PLL** vermenigvuldigers en **prescalers** omgezet worden naar 48 MHz, de hoogste snelheid waarop de STM32F091RC kan werken.

Na het opstarten van de microcontroller moet de code voorzien worden om de 48 MHz klok te gaan instellen.

Een mogelijkheid om dit te bekomen wordt hieronder aangegeven:

```
void SystemClock_Config(void)
{
    RCC->CR |= RCC_CR_HSI trimming_4;
    RCC->CR |= RCC_CR_HSION;
    while((RCC->CR & RCC_CR_HSIRDY) == 0);

    RCC->CFGR &= ~RCC_CFGR_SW;
    while((RCC->CFGR & RCC_CFGR_SWS) != RCC_CFGR_SWS_HSI);

    RCC->CR &= ~RCC_CR_PLLON;
    while((RCC->CR & RCC_CR_PLLRDY) != 0);

    RCC->CFGR |= RCC_CFGR_PLLSRC_HSI_PREDIV;
    RCC->CFGR2 |= RCC_CFGR2_PREDIV_DIV2;
    RCC->CFGR |= RCC_CFGR_PLLMUL12;

    FLASH->ACR |= FLASH_ACR_LATENCY;

    RCC->CR |= RCC_CR_PLLON;
    while((RCC->CR & RCC_CR_PLLRDY) == 0);

    RCC->CFGR |= RCC_CFGR_SW_PLL;
    while((RCC->CFGR & RCC_CFGR_SWS) != RCC_CFGR_SWS_PLL);

    RCC->CFGR |= RCC_CFGR_HPRE_DIV1;
    RCC->CFGR |= RCC_CFGR_PPRE_DIV1;

    SystemCoreClockUpdate();
    SysTick_Config(48000);
}
```

Figuur 29: codefragment met de klokinstellingen

Vragen:

- Analyseer deze code goed en zoek uit wat alle registers betekenen, hoe de bitbewerkingen effect hebben, ... Leg ook de figuur van de *clock tree* ernaast om dit te volgen.

Ga hier zeker niet licht over! Dergelijke code zal steeds opnieuw voorkomen als we de ARM microcontrollers gaan programmeren. Dit niet begrijpen is geen optie...